Format Roadmap Tool & Teknologi — Ketentuan Penyusunan Roadmap Alat/Teknologi

> **Dokumen ini adalah KETENTUAN untuk AI.** Dipakai saat menyusun roadmap sebuah **tool/teknologi non-bahasa pemrograman**.
> **Hasil akhir:** satu file `Roadmap/<Kategori>/Roadmap <Tool>.md`.
> **Struktur umum** (Filosofi, Prinsip Desain, level, benang merah, checkpoint) **mengacu ke** `Format Roadmap.md` — dokumen ini hanya mengatur **perbedaan & kurikulum inti tool/teknologi**.
> **Untuk framework (Laravel, React, NestJS): pakai** `Format Roadmap Framework.md`.
> **Untuk bahasa pemrograman:** `Format Roadmap.md`.

---

0. Kapan Dipakai & Beda dengan Roadmap Lain

Dipakai untuk alat/teknologi yang **bukan bahasa pemrograman dan bukan framework aplikasi**. Contoh:

Kelompok	Contoh	Catatan
Version Control	Git, GitHub/GitLab	tool wajib semua developer
Container & Infra	Docker, Docker Compose, Kubernetes, Nginx	deployment & environment
Basis Data	SQL, MySQL, PostgreSQL, Redis	bahasa query + tool
Markup & Styling	HTML, CSS, Sass	bukan bahasa pemrograman (tidak ada logika penuh)
Terminal & OS	Terminal/CLI, Bash, Linux	dasar kerja developer
Tooling & Kualitas	Regex, Postman, Vitest/Jest config, ESLint, Prettier	pendukung
CI/CD & Cloud	GitHub Actions, Vercel, VPS, AWS	otomatisasi & hosting

> **Batas penting:**
>
> - Tailwind & Bootstrap → **framework** (`Format Roadmap Framework.md`), bukan dokumen ini.
> - Node.js, PHP, Python, JavaScript → **bahasa** (`Format Roadmap.md`).

	Roadmap Bahasa	Roadmap Framework	Roadmap Tool/Teknologi (dokumen ini)
Kurikulum inti	21 fundamental bahasa	20 kurikulum framework	**Kurikulum tool** (Bagian 3)
Prasyarat	HTML/CSS, logika	bahasa dasarnya	**bervariasi** (lihat kurikulum)
Fokus	sintaks & logika	membangun aplikasi	**menguasai alat & workflow**
Praktik	satu project tumbuh	satu aplikasi nyata	**satu skenario nyata** (repo nyata, server nyata, deployment nyata)

---

1. Prinsip Wajib Roadmap Tool/Teknologi

Selain prinsip universal di `Format Roadmap.md` Bagian 1, tambahkan:

#	Prinsip Tambahan	Artinya
8	**Belajar dengan Tangan Kotor**	Setiap materi dipraktikkan pada kasus nyata (repo Git nyata, container nyata), bukan sekadar teori.
9	**Cheatsheet Dulu, Detail Kemudian**	Kumpulkan perintah/konsep inti lebih dulu, baru jelaskan mekanismenya.
10	**Integrasi Nyata**	Setiap tool diakhiri dengan cara mengintegrasikannya ke alur kerja developer (mis. Git + GitHub + CI).
11	**Satu Skenario Nyata**	Bangun satu skenario utuh dari nol sampai selesai. Contoh: repo dari `git init` → branch → PR → merge; atau aplikasi dari manual → Docker → deploy.
12	**Troubleshooting Wajib**	Sertakan error umum & cara mengatasinya. Tool tanpa troubleshooting tidak selesai.

---

2. Struktur Baku File Roadmap Tool/Teknologi

Struktur heading sama dengan `Format Roadmap.md` Bagian 2. Perbedaan (delta):

2.1 Kepala file — tambahan wajib

```
# 🗺️ Roadmap <Tool>: <Subjudul>

## Filosofi Roadmap Ini
> **"<kutipan>"** — <alasan>

### Prinsip Desain
- **Satu Skenario, Tuntas**: <skenario nyata yang dikerjakan>
- **Praktik, Bukan Teori**: <...>
- **Benang Merah Eksplisit**: ...

### Prasyarat Sebelum Memulai  ← TAMBAHAN
```

Sebelum roadmap ini, pastikan sudah:
├── <prasyarat 1, mis. menguasai terminal dasar>
├── <prasyarat 2>
└── <tool terpasang: git/docker/mysql + cara cek versinya>

```
### Spesifikasi Skenario  ← TAMBAHAN
| Aspek | Keterangan |
|---|---|
| Nama Skenario | <mis. "Repository Portofolio dengan Git + GitHub" |
| Deskripsi | <1–2 kalimat> |
| Yang Dibangun/Dilakukan | <daftar> |
| Alat Pendukung | <tool + versi> |
| Target Akhir | <hasil nyata yang terlihat> |
```

2.2 Gambaran besar

Alur level khas tool (contoh Git):

```
Level 1: Konsep & setup — apa itu VCS, instalasi, config, repo pertama
    ↓
Level 2: Alur dasar — add, commit, status, log, diff
    ↓
Level 3: Branching — branch, merge, conflict
    ↓
Level 4: Remote — clone, push, pull, PR, fork
    ↓
Level 5: Lanjutan — rebase, stash, tag, cherry-pick
    ↓
Level 6: Workflow tim — Git Flow, commit convention, hooks, CI
```

2.3 Tiap level — tambah baris "Praktik Nyata" & "Perintah Kunci"

```
## 🟢 LEVEL N: <TEMA> (Minggu a-b)

> **Tema**: _<...>_
> **Benang Merah**: <...>
> **Output**: <...>
> **Praktik Nyata**: <apa yang dikerjakan tangan di level ini>
> **Perintah Kunci**: `<perintah>`, `<perintah>`

---

### A. <Nama Bagian>

> 💡 **Mengapa dimulai di sini?** <alasan>
```

Benang Merah Bagian A:
<...> → <...> → <...>

```
1. `[[<Nomor>. <Judul Materi Tool>]]`

    - <poin>
    - **Cheatsheet**: `<perintah1>` → <fungsi>; `<perintah2>` → <fungsi>
    - **Error umum**: <gejala> → <solusi>
    - _Langkah konkret_: <aksi di terminal/aplikasi>
```

2.4 Checkpoint — tambah blok TROUBLESHOOTING

```
### 🪜 Checkpoint Level N
```

✅ Checklist sebelum lanjut ke Level N+1:

PEMAHAMAN:
├── Bisa jelaskan
└── ...

PRAKTIK:
├── <perintah/skenario yang harus berhasil>
└── ...

TROUBLESHOOTING:
├── Bisa mengatasi <error umum 1>
└── Bisa mengatasi <error umum 2>

---

3. Kurikulum Inti Tool/Teknologi (WAJIB tercakup)

3.1 Kerangka generik (berlaku semua tool)

Setiap roadmap tool minimal membahas:

No	Materi	Yang harus dibahas
1	**Konsep & Filosofi Alat**	Masalah yang dipecahkan, kapan dipakai, alternatifnya.
2	**Instalasi & Setup**	Cara pasang, cek versi, konfigurasi awal.
3	**Konfigurasi**	File config, environment, best practice default.
4	**Perintah/Konsep Inti**	Cheatsheet operasi paling sering dipakai.
5	**Alur Kerja Dasar**	Rangkaian perintah/konsep dalam satu tugas nyata.
6	**Fitur Lanjutan**	Fitur yang membedakan pemula dan mahir.
7	**Integrasi dengan Tool Lain**	Cara masuk ke alur kerja developer secara keseluruhan.
8	**Troubleshooting**	Error umum & solusi.
9	**Best Practice & Keamanan**	Kebiasaan aman & efisien.
10	**Skenario Nyata Tuntas**	Proyek/skenario dari nol sampai selesai.

3.2 Kurikulum spesifik per tool (pakai yang relevan)

Git
Konsep VCS → instalasi & config → repo (`init` / `clone`) → alur dasar (`status` / `add` / `commit` / `log` / `diff`) → `.gitignore` → branching → merge & conflict → remote (`push` / `pull` / `fetch`) → Pull Request & fork → rebase → stash → tag & release → cherry-pick → reset/revert/restore → hooks → commit convention → Git Flow.

Docker
Konsep & arsitektur → instalasi → image vs container → perintah dasar

(`run` / `ps` / `exec` / `stop` / `rm` / `logs`) → Dockerfile → build & tag → volume → network → Docker Compose (multi-container) → registry (`push` / `pull`) → multi-stage build → environment & secrets → best practice image → deployment.

SQL / MySQL / PostgreSQL
Konsep basis data & RDBMS → instalasi & klien → `CREATE DATABASE` / `USE` → tipe data → DDL (`CREATE` / `ALTER` / `DROP`) → DML (`INSERT` / `UPDATE` / `DELETE`) → `SELECT` + `WHERE` / `ORDER` / `LIMIT` → fungsi agregat & `GROUP BY` / `HAVING` → JOIN (inner/left/right/full) → subquery → index → relasi & foreign key → normalisasi → view & stored procedure → transaksi (`BEGIN` / `COMMIT` / `ROLLBACK`) → user & grant → backup & restore.

HTML / CSS
Struktur dokumen HTML → elemen & semantik → atribut → link & gambar → tabel → form & input → aksesibilitas dasar → CSS: selector & spesifisitas → box model → warna & tipografi → unit (`px` / `rem` / `%` / `vw`) → display & positioning → Flexbox → Grid → responsive & media query → transisi & animasi → variabel CSS → best practice.

Terminal / Linux
Navigasi (`pwd` / `cd` / `ls`) → file (`cp` / `mv` / `rm` / `mkdir` / `touch`) → melihat isi (`cat` / `less` / `head` / `tail`) → pencarian (`grep` / `find`) → pipe & redirect → permission (`chmod` / `chown`) → user & group → proses (`ps` / `top` / `kill`) → paket manager (`apt` / `dnf`) → environment variable → SSH → cron.

Regex
Literal & metakarakter → character class → quantifier → anchor → grouping & alternation → capture group → lookaround → flag → penggunaan di bahasa/tool → kasus nyata (validasi, parsing).

CI/CD (GitHub Actions dsb.)
Konsep pipeline → workflow & event trigger → job & step → runner → build → test otomatis → deploy otomatis → secret & environment → matrix → cache → notifikasi.

> **Catatan adaptasi:** ambil poin yang relevan; untuk tool sederhana (mis. Postman) cukup Bagian 3.1. Selalu sertakan **Troubleshooting** dan **Skenario Nyata Tuntas**.

---

4. Contoh Alur Level

Docker

```
Level 1: Konsep & instalasi, image vs container, perintah dasar
Level 2: Dockerfile, build, tag
```

```
Level 3: Volume & network, data persisting
Level 4: Docker Compose (app + db + nginx)
Level 5: Registry, multi-stage build, secrets
Level 6: Best practice image & deployment
```

SQL/MySQL

```
Level 1: Konsep RDBMS, instalasi, database & tabel, tipe data
Level 2: DML (CRUD), SELECT, WHERE, ORDER
Level 3: Agregasi, GROUP BY, JOIN
Level 4: Relasi, foreign key, index, normalisasi
Level 5: View, stored procedure, transaksi
Level 6: User, hak akses, backup & restore
```

---

5. Aturan Penomoran, Link, dan Konsistensi

Sama seperti `Format Roadmap.md` Bagian 6, dengan penekanan:

1. Nomor materi **berurutan global** lintas level.
2. Setiap poin memuat **cheatsheet** (perintah/konsep inti) dan **error umum**.
3. Link materi: `` `[[<Nomor>. <Judul> — <Subjudul>]]` `` (tanpa path, aman karena nama unik).
4. Sebutkan **versi tool** (mis. Git 2.45, Docker 27, MySQL 8) supaya materi tidak basi.
5. Bedakan jelas: **tool** (belajar memakai) vs **framework** (belajar membangun aplikasi).

---

6. Template Roadmap Tool/Teknologi (copy-paste)

```
# 🗺️ Roadmap <Tool>: <Subjudul>

## Filosofi Roadmap Ini

> **"<kutipan>"** — <alasan>

### Prinsip Desain

- **Satu Skenario, Tuntas**: <...>
- **Praktik, Bukan Teori**: <...>
- **Benang Merah Eksplisit**: setiap langkah terhubung ke langkah sebelum dan sesudahnya
```

```
### Prasyarat Sebelum Memulai
```

Sebelum roadmap ini, pastikan sudah:
├── <prasyarat 1>
└── <tool terpasang + cara cek versi>

```
### Spesifikasi Skenario

| Aspek | Keterangan |
|---|---|
| Nama Skenario | <...> |
| Deskripsi | <...> |
| Yang Dibangun/Dilakukan | <...> |
| Alat Pendukung | <...> |
| Target Akhir | <...> |

---

## 📋 Gambaran Besar — Apa yang Akan Dikuasai
```

Level 1: ...
↓
Level N: ...

```
---

## 🟢 LEVEL 1: <TEMA> (Minggu 1–x)

> **Tema**: _<...>_
> **Benang Merah**: <...>
> **Output**: <...>
> **Praktik Nyata**: <...>
> **Perintah Kunci**: `<...>`, `<...>`

---

### A. <Bagian>
```

```
> 💡 **Mengapa dimulai di sini?** <...>
```

Benang Merah Bagian A:
<...> → <...> → <...>

```
1. `[[1. <Judul> — <Subjudul>]]`

    - <poin>
    - **Cheatsheet**: `<perintah>` → <fungsi>
    - **Error umum**: <gejala> → <solusi>
    - _Langkah konkret_: <aksi>

---

### 🧱 Checkpoint Level 1
```

✅ Checklist sebelum lanjut ke Level 2:

PEMAHAMAN:
├── <...>

PRAKTIK:
├── <...>

TROUBLESHOOTING:
├── <...>

```
```

---

7. Checklist Kualitas Roadmap Tool/Teknologi

- [ ] Ada **Prasyarat** dan **Spesifikasi Skenario**.
- [ ] Kerangka generik (Bagian 3.1) tercakup.
- [ ] Kurikulum spesifik tool (Bagian 3.2) tercakup atau dijelaskan yang tidak relevan.
- [ ] Setiap poin punya **cheatsheet** dan **error umum**.
- [ ] Ada bagian **Troubleshooting** nyata.
- [ ] Checkpoint memuat blok **TROUBLESHOOTING**.
- [ ] Ada satu **skenario nyata tuntas** dari nol sampai selesai.

- [ ] Versi tool disebutkan.
- [ ] Jelas beda antara tool dan framework.

File terkait:

- `Format Roadmap.md` — struktur umum & roadmap bahasa pemrograman.
- `Format Roadmap Framework.md` — roadmap framework.
- `Format Penjelasan Materi.md` — menulis materi tiap poin.
- `Format Pembuatan Project.md` — menyusun roadmap project.